Recitation 4 – Socket Programming, Mail, and DNS

Daniel Karalnik and Liam Tal

Computer Networks

Table of Contents

1. Socket Programming
2. Mail Protocols
3. DNS

Today

- Socket Programming
- Mail Protocols
- DNS

Focus: Application layer protocols and practical network programming

Recap: Socket

What's a socket?

Hint: Recall Operating Systems course...

Recap: Socket

What's a socket?

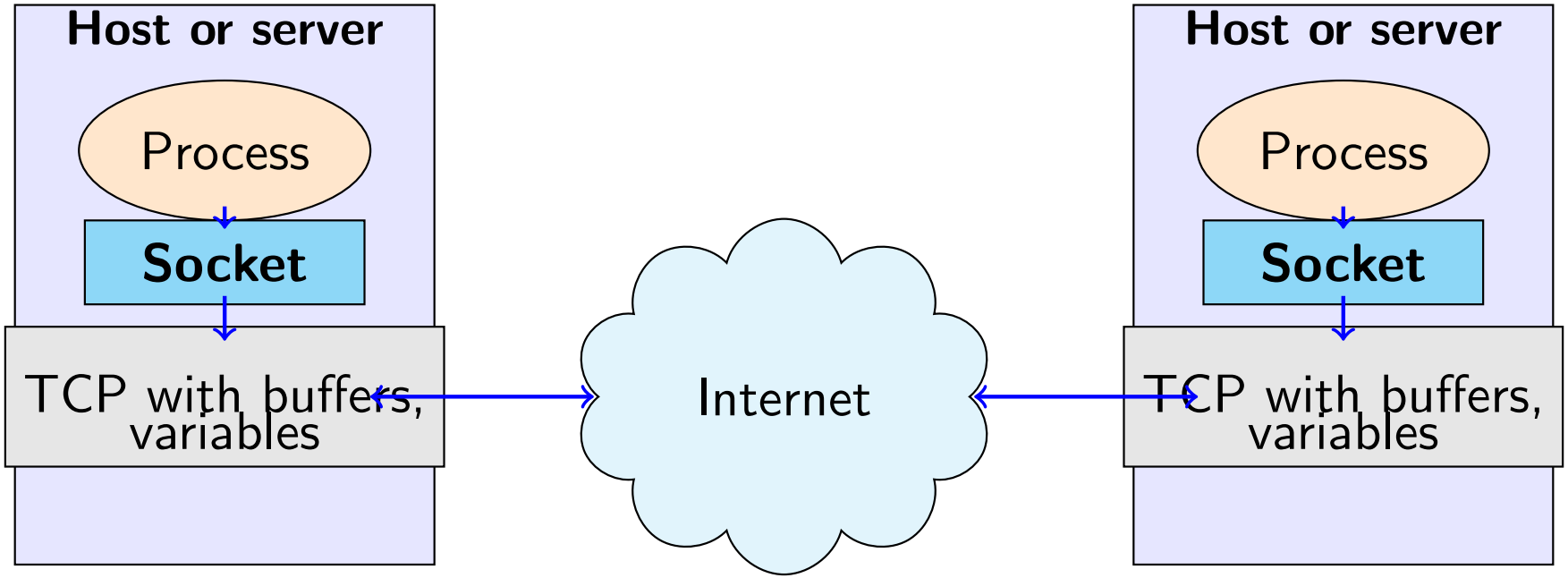
Hint: Recall Operating Systems course...

Definition

A socket is the interface between a process and the network.
It's the endpoint for sending and receiving data across a network. Think of it as a "door" through which applications communicate with the network stack—just as a door is a specific point of entry/exit in a building, a socket is the specific point where data enters and exits your application.

Recap: Socket - Visual

The interface between a process and the network.



Socket Types - UDP and TCP

Main socket types for network programming:

UDP Socket - Datagram Socket

Characteristics:

- Connectionless - No handshake before sending data
- No reliability guarantees (packets may be lost/reordered)
- Message-oriented - Preserves message boundaries
- Fast and lightweight (compared to TCP)
- No flow control or congestion control

Socket Types - UDP and TCP

Main socket types for network programming:

TCP Socket - Stream Socket

Characteristics:

- Connection-oriented - Three-way handshake required
- Reliable - Guaranteed delivery with acknowledgments
- Stream-oriented - No message boundaries
- Ordered delivery - Packets arrive in sequence
- Flow control and congestion control built-in

Socket Types - Raw Sockets and System Calls

Raw Socket

Direct packet access - Receive/send entire packet, including headers

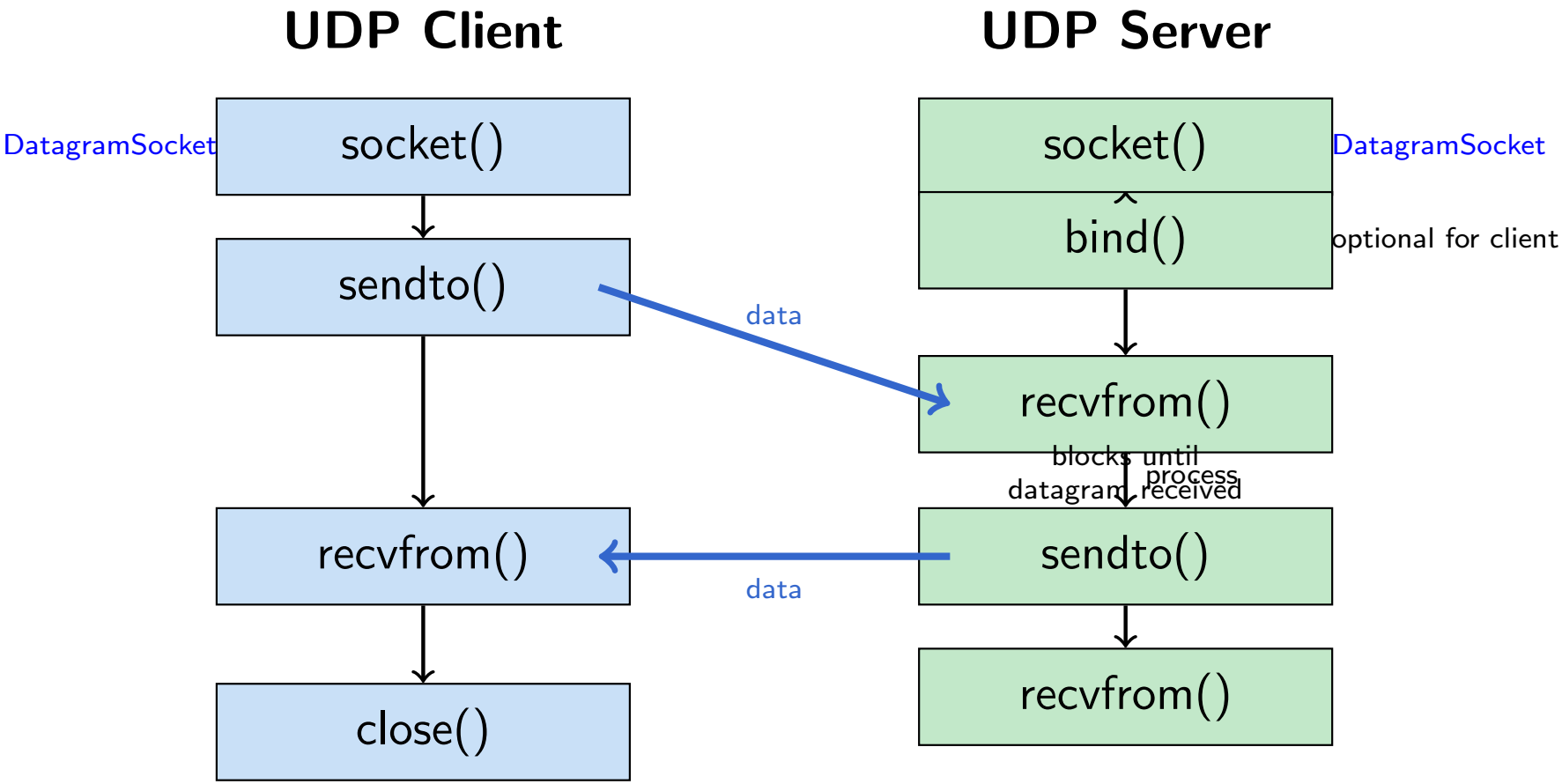
Use cases:

- Network monitoring tools (packet sniffers)
- Custom protocol implementation
- Low-level network diagnostics
- Security tools (ping, traceroute)

Requirements:

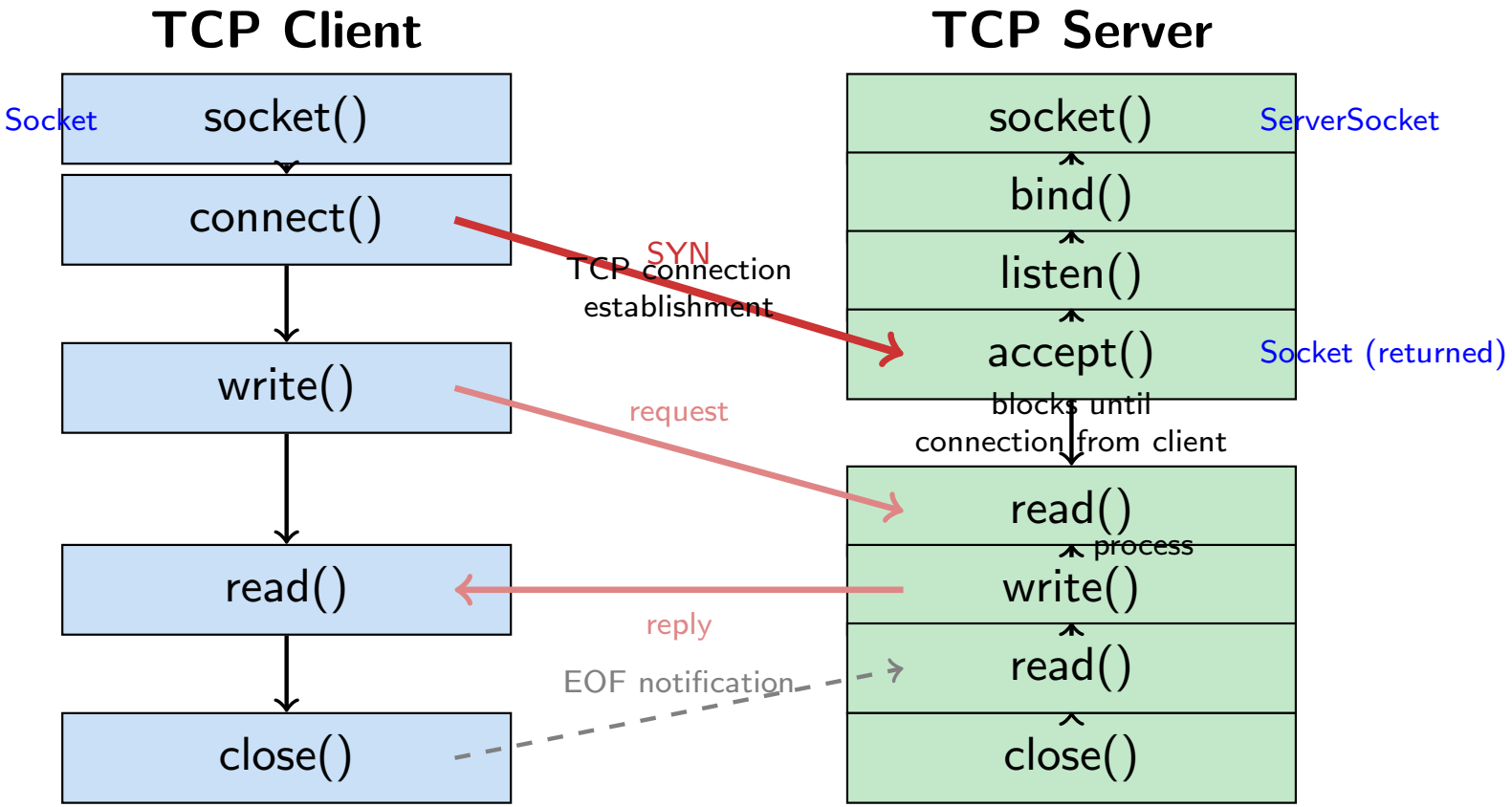
- Requires special privileges (root/administrator access)
- Bypasses transport layer processing
- Not covered in this course

UDP Socket Lifecycle



Key Point: UDP is connectionless - no handshake, just send and receive datagrams

TCP Socket Lifecycle



TCP Socket Lifecycle (continued)

Key Differences from UDP

- **Connection establishment:** Three-way handshake before data transfer
- **Server accepts:** Creates new socket for each client connection
- **Stream-oriented:** Use read()/write() system calls (in Java: `getInputStream()`/`getOutputStream()`)
- **Connection termination:** Proper shutdown with close()

Java API

Client: `Socket`
Server: `ServerSocket` (for listening) + `Socket` (for each connection)
I/O: Use `getInputStream()` and `getOutputStream()`

Handling Concurrent Connections

The Challenge: How do we handle multiple clients simultaneously?

The Problem: Blocking Operations

Network operations (accept, read, write) are **blocking** by default:

- Server calls `accept()` $\rightarrow$ blocks until client connects
- Server calls `read()` $\rightarrow$ blocks until data arrives
- While blocked, cannot handle other clients!

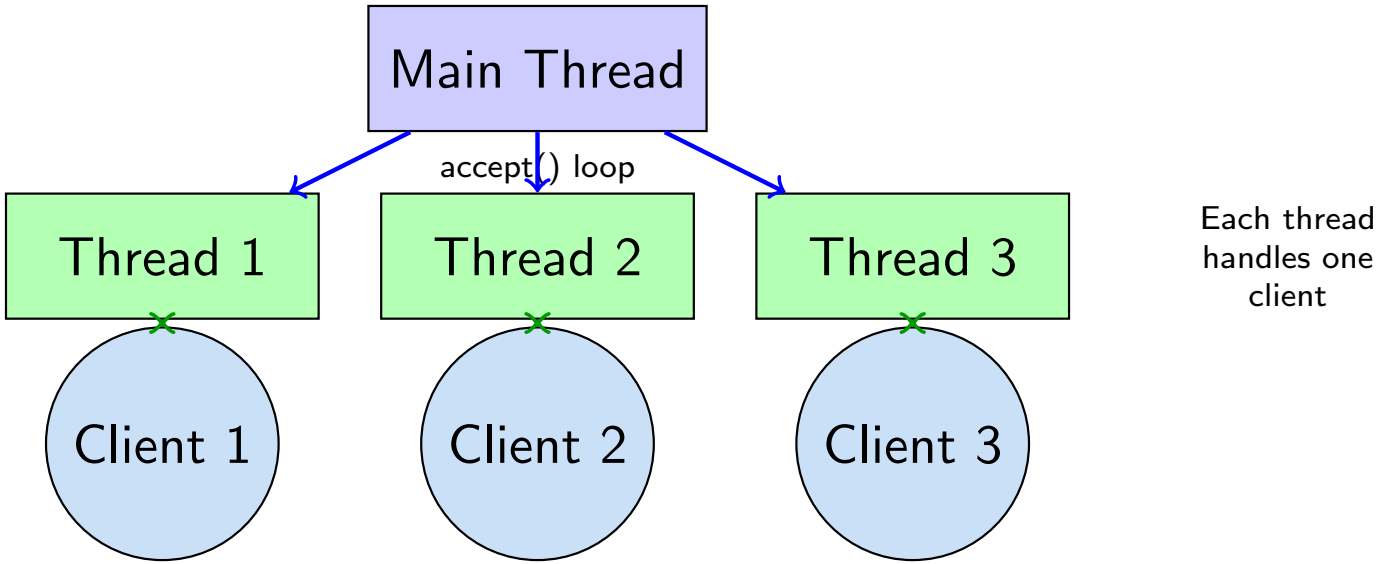
Two Main Solutions:

Multi-threaded
One thread per client

Asynchronous (Async)
Single thread, event-driven

Multi-threaded Approach - Architecture

Concept: Create a new thread for each client connection (or use a thread pool)



Main thread accepts connections and spawns worker threads. Each worker thread independently handles its client's requests.

Multi-threaded Approach - Trade-offs

Advantages

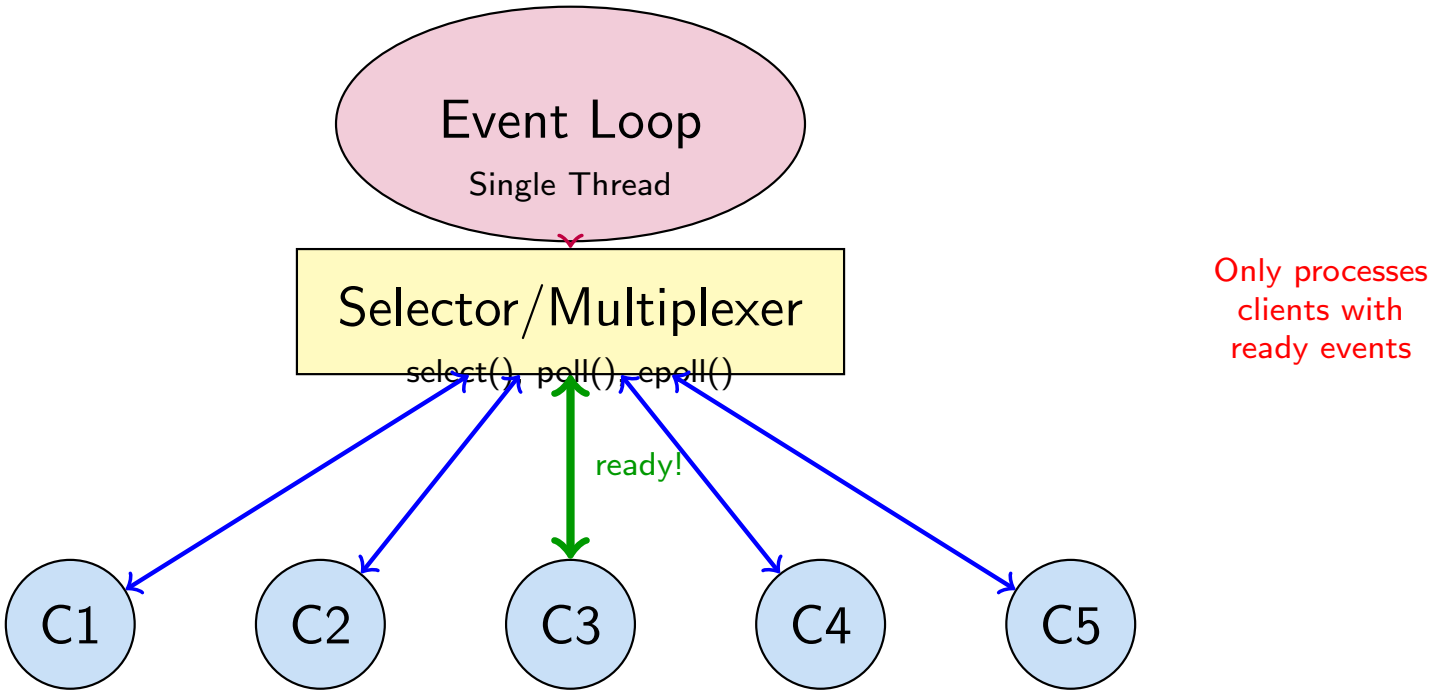
- Simple to understand
- Easy to implement
- Natural blocking I/O
- Good for CPU-bound tasks
- Thread pools limit overhead

Disadvantages

- Thread overhead (memory, CPU)
- Context switching cost
- Limited scalability (hundreds to thousands)
- Thread synchronization complexity

Asynchronous (Async) Approach - Architecture

Concept: Single thread monitors all connections, handles events as they occur



Event loop uses a selector to monitor many clients. When data is ready, it processes that client and returns to monitoring.

Asynchronous (Async) Approach - Trade-offs

Advantages

- Low memory overhead
- Scales to many clients
- No context switching
- Efficient for I/O-bound tasks

Disadvantages

- More complex programming
- Callback hell / complexity
- One CPU-blocking operation stalls all clients
- Not ideal for CPU-bound work

Comparison: Multi-threaded vs Async

Aspect	Multi-threaded	Async
Programming Model	Sequential, intuitive	Event-driven, callbacks
Scalability	Hundreds to thousands	Tens of thousands+
Resource Usage	High (stack per thread, context switching)	Low (shared stack)
Best Use Case	CPU-bound tasks	I/O-bound tasks
Debugging	Easier	Harder

Key Insight

Choose based on your workload:

- **Multi-threaded:** CPU-bound tasks, moderate concurrency, simpler code
- **Async:** I/O-bound tasks, high concurrency (thousands of connections)

Asynchronous Programming Technologies - Part 1

How different languages/frameworks handle async concurrency:

Java

- **NIO (New I/O):** Non-blocking I/O with buffers and channels
- `Selector` - Monitors multiple channels for events
- `Channel` - Non-blocking read/write operations
- **Virtual threads (Project Loom):** Lightweight threads for async (Java 19+)

Asynchronous Programming Technologies - Part 2

How different languages/frameworks handle async concurrency:

Python

- `asyncio` module - Event loop and coroutines
- `async/await` syntax - Define and await async functions
- Great for I/O-bound tasks (network, file operations)
- Works well with libraries like `aiohttp`, `FastAPI`

Asynchronous Programming Technologies - Part 3

More async concurrency frameworks:

JavaScript/Node.js

- Event loop (built-in) - Core of Node.js architecture
- Promises - Represent future values, handle async results
- `async/await` - Cleaner async syntax (ES2017+)
- Callback-based APIs (older style)

Important Notes

System Ports Security

On Linux/macOS, only root can bind to privileged ports (1-1023)

Why?

- Security measure to prevent unprivileged users from impersonating system services
- Well-known ports (HTTP=80, HTTPS=443, SSH=22, etc.) are protected
- Prevents malicious programs from hijacking important services
- Port 0 is special: means "assign any available port"

Mail Protocols - Quick Recap

From the lecture, which 3 mail protocols were discussed?

Mail Protocols - Quick Recap

From the lecture, which 3 mail protocols were discussed?

- SMTP (Simple Mail Transfer Protocol)
- POP3 (Post Office Protocol version 3)
- IMAP (Internet Message Access Protocol)

Which additional method to send/receive mail was discussed?

Mail Protocols - Quick Recap

From the lecture, which 3 mail protocols were discussed?

- SMTP (Simple Mail Transfer Protocol)
- POP3 (Post Office Protocol version 3)
- IMAP (Internet Message Access Protocol)

Which additional method to send/receive mail was discussed?

- Web Mail (HTTP/HTTPS based)

Mail Protocols - Interactive Quiz

Let's test your knowledge!

Protocol	Deliver / Retrieve?	Encryption?	Port?
SMTP	?	?	?
POP3	?	?	?
IMAP	?	?	?
Web Mail	?	?	?

Think about: What is each protocol used for? Does it support encryption? What ports does it use?

Mail Protocols - Comparison Table

Protocol	**Deliver / Retrieve?**	**Encryption?**
SMTP	Deliver	Optional
POP3	Retrieve	Optional
IMAP	Retrieve (and manipulate)	Optional
Web Mail	Both	Yes

Mail Protocols - Comparison Table

Key Differences

- **SMTP:** Only for *sending/delivering* mail (push protocol)
- **POP3/IMAP:** For *retrieving* mail from server (pull protocols)
- **POP3:** Downloads and typically deletes from server (offline access)
- **IMAP:** Keeps mail on server, supports synchronization across devices
- **Web Mail:** Both send and receive via HTTP/HTTPS in a browser

Mail Protocols - SMTP

SMTP - Simple Mail Transfer Protocol

Purpose: Send/deliver emails
Direction: Client $\rightarrow$ Server, Server $\rightarrow$ Server
Ports:

- Port 25: Server-to-server communication (MTA to MTA)
- Port 587: Client submission to server (MSA) with STARTTLS encryption
- Port 465: Legacy SMTP over SSL (deprecated, but still used)

Usage: Your email client uses SMTP to send outgoing messages to your email provider's server.

Mail Protocols - POP3 & IMAP

POP3 - Post Office Protocol v3

Purpose: Download emails from server

Behavior:

- Downloads messages to local device
- Typically deletes from server after download
- Good for offline access
- No folder synchronization

IMAP - Internet Message Access Protocol

Purpose: Retrieve and manage emails on server

Behavior:

- Keeps mail on server
- Supports folders and flags
- Syncs across multiple devices
- Modern standard for email access

Domain Name System (DNS)

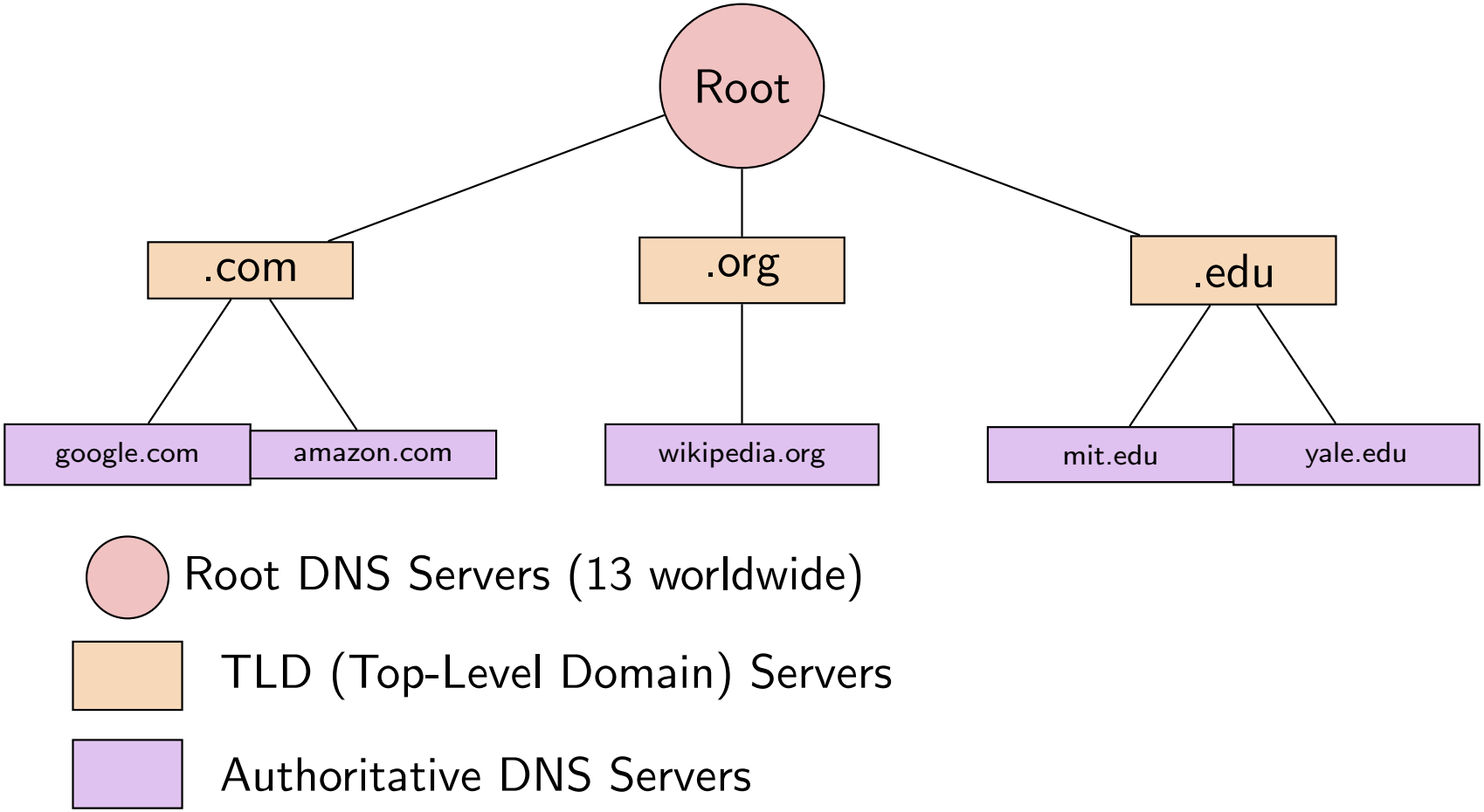
What is DNS?

The Domain Name System is a hierarchical, distributed database that translates human-readable domain names (like `www.example.com`) into IP addresses (like `93.184.216.34`).

Key Characteristics:

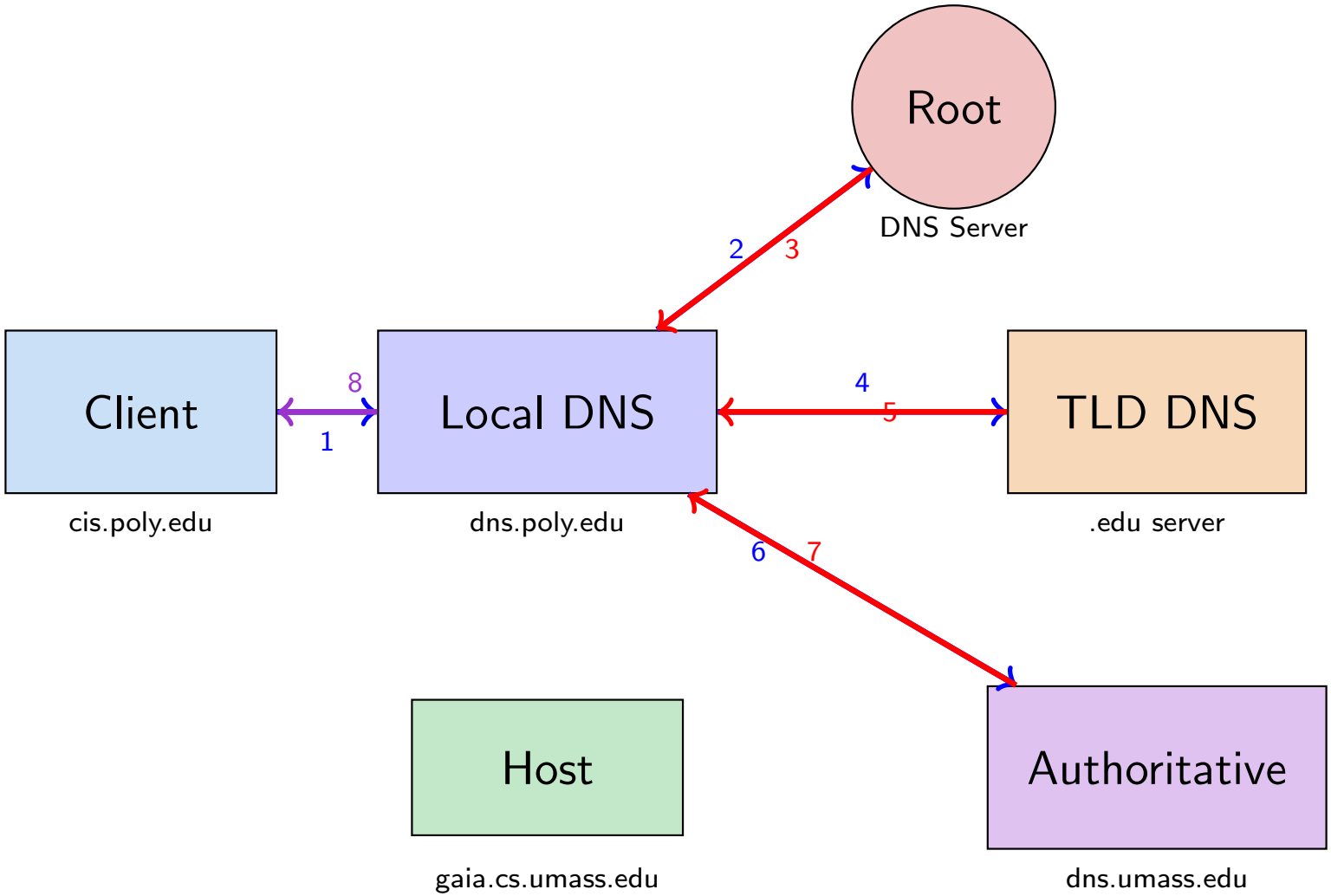
- **Application Layer** protocol
- **Query types:**
  - *Recursive*: Client → Local DNS (client asks DNS to do all the work)
  - *Iterative*: Local DNS → Other servers (DNS server asks and gets referrals)
- **Transport:** Primarily UDP port 53; TCP for responses
- **Hierarchical:** Root → TLD → Authoritative servers
- **Caching:** TTL (Time To Live) specifies cache duration

DNS Hierarchy



Distributed Design: No single point of failure, scalable, and manageable by different organizations

DNS Query Flow



DNS Query Flow

Query Steps

Steps: 1. Client queries local DNS $\rightarrow$ 2-3. Local queries root $\rightarrow$ 4-5. Local queries TLD $\rightarrow$ 6-7. Local queries authoritative $\rightarrow$ 8. Answer returned to client

Caching Impact: In practice, most queries are answered from cache, dramatically reducing load on root/TLD servers. TTL values balance freshness vs. query load.

DNS Record Types - Address & Lookup Records

A Record (Address)

Purpose: Maps hostname → IPv4 address
Format: hostname → IPv4 (32-bit)
Example: `example.com` → `93.184.216.34`

Most common DNS record type for web browsing

AAAA Record (IPv6 Address)

Purpose: Maps hostname → IPv6 address
Format: hostname → IPv6 (128-bit)
Example: `example.com` → `2001:db8::1`

"Quad-A" record - four times as many bits as A record

DNS Record Types - Service & Infrastructure Records

CNAME Record (Canonical Name)

Purpose: Creates an alias from one hostname to another
Format: alias → canonical hostname
Example: `www.example.com → example.com`

Useful for load balancing and CDNs

MX Record (Mail Exchange)

Purpose: Specifies mail server for a domain
Format: domain → mail server + priority number
Example: `example.com → mail.example.com (priority: 10)`

DNS Record Types - Additional Types

TXT Record (Text)

Purpose: Stores arbitrary text data
Use cases:

- Domain verification (Google, Microsoft, etc.)
- General notes and metadata

SOA Record (Start of Authority)

Purpose: Contains administrative information about the zone
Information includes:

- Primary nameserver for the zone (domain)
- Other metadata (like contact email etc)

DNS Packet Structure - Query

DNS Query Packet

Transaction ID	Flags
Question count	Answer RR count
Authority RR count	Additional RR count
Question entries (variable length)	
Additional sections (empty in query)	

Key Fields: Flags (Request/Response, Recursion Desired/Available, Return Code) — Question (Name, Type, Class)

DNS Packet Structure - Response

DNS Response Packet

Transaction ID	Flags
Question count	Answer RR count
Authority RR count	Additional RR count
Question entries (variable)	
Answer RRs (variable)	
Authority RRs (variable)	
Additional RRs (variable)	

Resource Record Format

Fields: Name (variable) — Type (16 bits) — Class (16 bits) — TTL (32 bits) — Data length (16 bits) — Data (variable)

dig - DNS Query Tool

What is dig?

dig is a powerful command-line tool developed by BIND for querying DNS nameservers. It provides detailed information about DNS queries and responses.

Installation:

- **macOS / Linux:** Available out-of-the-box
- **Windows users:**
  - Download BIND: `https://www.isc.org/downloads/`
  - Add to PATH environment variable

Why dig? Much more powerful than `nslookup` - shows detailed query/response information, supports all record types, and offers fine-grained control over queries.

dig - Common Commands

Command	Description	Example
`dig [hostname]`	Returns any A record for hostname	`dig dyn.com`
`dig [hostname] [type]`	Returns records of specified type	`dig dyn.com MX`
`dig [hostname] +short`	Terse answer (just IP)	`dig dyn.com +short`
`dig @[server] [hostname]`	Query specific nameserver	`dig @8.8.8.8 dyn.com`
`dig [hostname] +trace`	Trace from root to authoritative	`dig dyn.com +trace`
`dig -X [IP]`	Reverse lookup	`dig -X 8.8.8.8`
`dig [hostname] any`	All records for hostname	`dig dyn.com any`

Tip: Use `+trace` to see the full resolution path from root $\rightarrow$ TLD $\rightarrow$ authoritative!

Iterative Query Demo - en.wikipedia.org

Manual DNS resolution using iterative queries:

Step 1: Get Root Servers

```
dig . NS @8.8.8.8
```

Returns all 13 root servers (a.root-servers.net through m.root-servers.net)

Step 2: Query Root for en.wikipedia.org

```
dig en.wikipedia.org +norecurse @a.root-servers.net
```

Root returns .org TLD nameservers

Step 3: Query .org TLD

Iterative Query Demo - en.wikipedia.org

Manual DNS resolution using iterative queries:

Step 1: Get Root Servers

```
dig . NS @8.8.8.8
```

Returns all 13 root servers (a.root-servers.net through m.root-servers.net)

Step 2: Query Root for en.wikipedia.org

```
dig en.wikipedia.org +norecurse @a.root-servers.net
```

Root returns .org TLD nameservers

Iterative Query Demo - en.wikipedia.org

Step 3: Query .org TLD

```
dig en.wikipedia.org +norecurse @a0.org.afilias-nst.info
```

TLD returns wikipedia.org authoritative nameservers

Step 4: Query Authoritative NS

```
dig en.wikipedia.org +norecurse @ns0.wikimedia.org
```

Finally get the IP address!

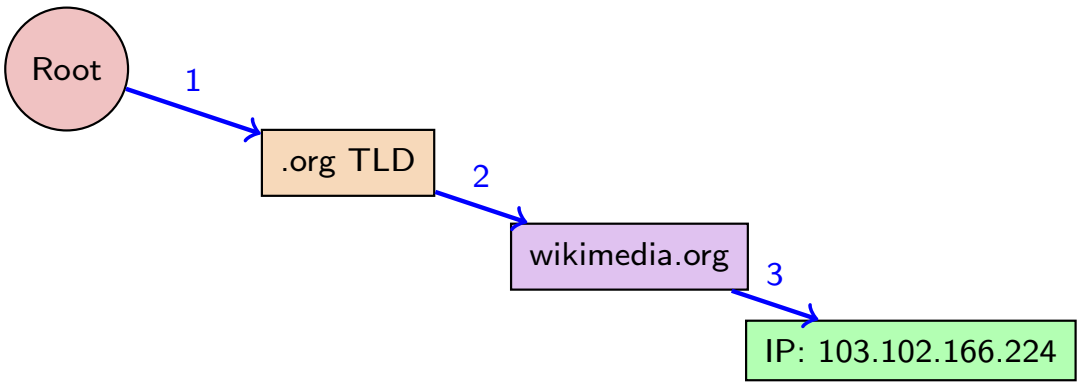
Iterative Query Demo - Continued

Or... Use the +trace Option!

```
dig en.wikipedia.org +trace
```

This automatically performs all the iterative queries and shows you the full path!

Note: Actual DNS servers may vary over time; this demonstrates the iterative resolution process.



DNS Security - Cache Poisoning

Historical vulnerabilities and mitigations:

1997: Bailiwick Checking

Problem: Attackers could inject additional records from different domains in the response
Solution (Bailiwick Checking): Only accept resource records that are within the authority ("bailiwick") of the queried domain
Example:

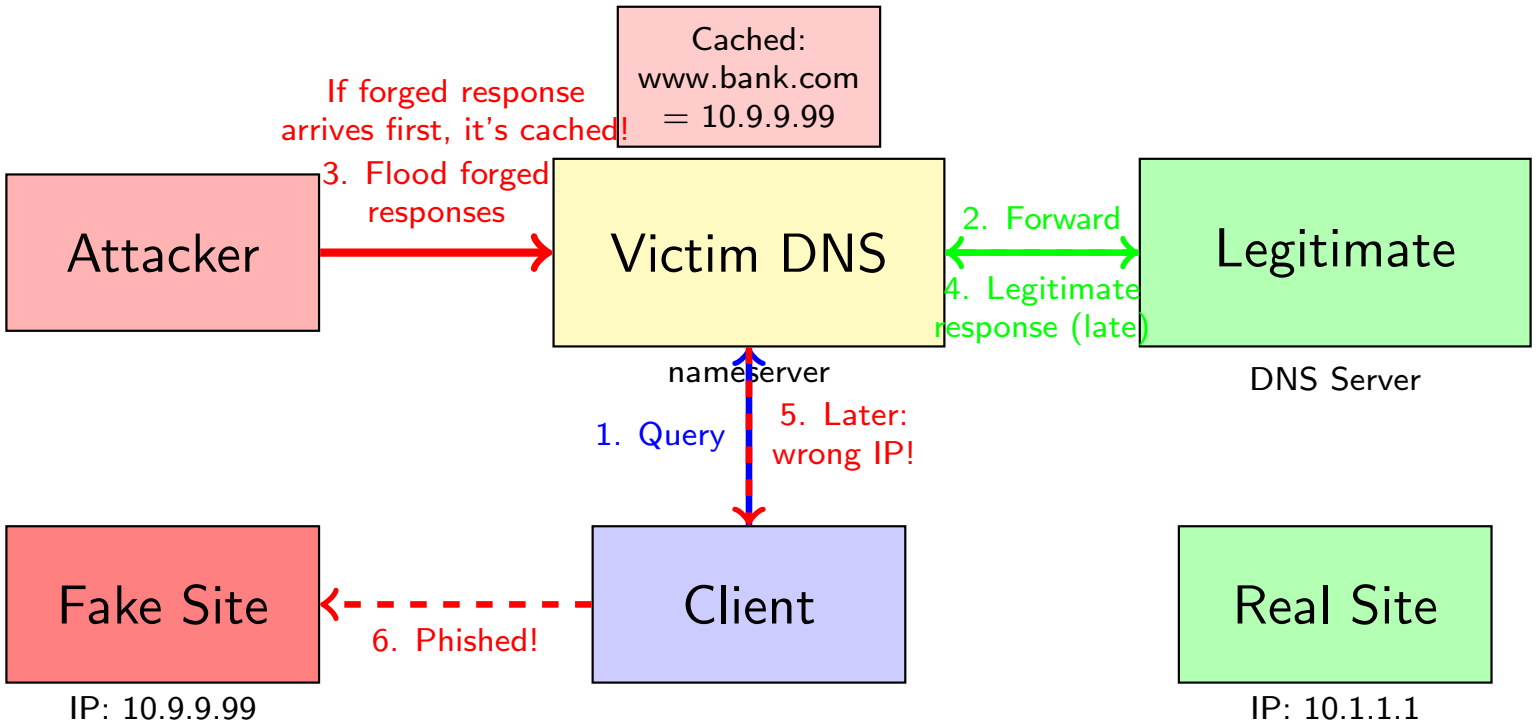
- Query for `www.bank.com`
- Accept answers about `*.bank.com` (within bailiwick)
- Reject answers about `www.attacker.com` (outside bailiwick)

DNS Security - Cache Poisoning

DNS Cache Poisoning Attack

Problem: Transaction IDs were predictable (often incremental/sequential)
Attack: Attacker sends many forged responses with guessed IDs, racing to respond before the legitimate DNS server
Mitigation: Randomize transaction IDs (16 bits), use source port randomization (+16 bits), implement DNSSEC

DNS Cache Poisoning - Attack Diagram



Reference: Understanding Kaminsky's DNS Bug

DNS Security - Modern Mitigations

Randomization Techniques

Transaction ID + Source Port Randomization:

- 16-bit random transaction ID (65,536 possibilities)
- 16-bit random source port (additional 65,536 possibilities)
- Combined: $2^{32} = 4.3$ billion combinations
- Makes cache poisoning attacks impractical within the small time window

DNS Security - Modern Mitigations

DNSSEC

- Cryptographic signatures
- Chain of trust from root
- Prevents cache poisoning
- Authenticates DNS data

Best Practices

- Use trusted DNS servers
- Enable DNSSEC validation
- Keep DNS software updated
- Monitor for anomalies

DNSSEC - DNS Security Extensions

What is DNSSEC?

DNSSEC adds cryptographic authentication to DNS responses, ensuring that the data you receive actually came from the authoritative source and hasn't been tampered with.

Digital Signatures

- Each DNS record is digitally signed
- Uses public/private key cryptography
- Signature proves authenticity

Chain of Trust

- Root DNS signs TLD keys
- TLD DNS signs domain keys
- Domain signs its records
- Validates entire path

DNSSEC - Benefits and Limitations

Benefits of DNSSEC

Key Security Improvements:

- **Prevents DNS cache poisoning:** Forged responses are detected and rejected
- **Authenticates DNS data:** Ensures responses come from authoritative source
- **Integrity verification:** Data hasn't been modified in transit
- **Building block for security:** Enables DANE, TLSA, and other security protocols

DNSSEC - Benefits and Limitations

Limitations of DNSSEC

Current Challenges:

- **No confidentiality:** Queries/responses still visible (combine with DNS over HTTPs/DNS over TLS for privacy)
- **Complex deployment:** Key management, rotation, and zone signing complexity
- **Larger responses:** Signatures add overhead (can exceed 512 bytes)
- **Not universally adopted:** Many domains still don't use DNSSEC